



BILDUNG. FREUDE INKLUSIVE.

Webdesign im IT-L@B

Einführung in HTML und CSS
Flex-Layouts mit CSS

Überarbeitet: 03.02.2025

von Dipl.-Ing. (FH) Robert Schoblick, M.Sc.



Kärntner Berufsförderungsinstitut GmbH
Bahnhofstraße 44
9020 Klagenfurt am Wörthersee
T. +43 (0)5 78 78
info@bfi-kaernten.at

www.bfi-kaernten.at



Die beste
Zeit für
Weiterbildung
ist JETZT!

Arbeitsauftrag:

1. **Lesen** Sie das Skript **vollständig**!
2. **Programmieren** Sie die vorgestellten Beispiele – auch in eigenen Varianten – nach. Vermeiden Sie es, fremde Programmskripte zu kopieren!
3. **Bearbeiten** Sie den am Ende der Lektion gestellten Arbeitsauftrag!
4. **Testen** Sie die Webseite auf verschiedenen Webbrowsern!

**Die Code-Beispiele sind als Grafik in dieses Dokument eingefügt.
D.h.: Copy&Paste ist keine mögliche Lösung!**

Arbeiten Sie NICHT mit ChatGPT oder anderen KI-Systemen, denn SIE sollen das Programmieren erlernen!!!!

Kalkulierte Bearbeitungszeit

Die Aufgabe ist so gestaltet, dass Sie einschließlich der Lektüre dieser Lektion, dem Ausprobieren der Beispiele und eventuell eigene Recherchen im Internet maximal **24 Stunden Arbeitszeit** benötigen sollten. Wenn Sie Probleme (z.B. Schwierigkeiten mit dem Verständnis der deutschen Sprache) haben sollten, wenden Sie sich bitte an Ihre Trainerin oder Ihren Trainer.

Der Inhalt dieses Moduls ist sehr umfangreich. Lesen Sie das Skript aufmerksam und bearbeiten Sie die Übungen so, dass Sie auch spätere Prüfungsfragen beantworten können.

Für die Abgabe der Aufgaben gelten folgende Regeln:

Offizielle Lehrlinge des IT-L@B:

Lehrlinge des IT-@B verfügen über einen Zugang zum Lernsystem Moodle. Rufen Sie den folgenden Kurs auf:

20E 55 - User Experience - IT-L@B Villach

Geben Sie die Arbeit im dafür vorgesehenen Abgabefeld der Aufgabe fristgerecht ab.

Praktikantinnen und Praktikanten:

Praktikantinnen und Praktikanten haben noch keinen Zugang zum Moodle-System. Sie geben die Aufgabe bitte in Ihrem **persönlichen Ordner** auf dem NAS („\\itlabvi\\public“) ab. Richten Sie dafür bitte ein Unterverzeichnis „web“ ein.

Inhalt

Arbeitsauftrag:	2
Kalkulierte Bearbeitungszeit	2
Für die Abgabe der Aufgaben gelten folgende Regeln:	2
Offizielle Lehrlinge des IT-L@B:.....	2
Praktikantinnen und Praktikanten:	2
Responsives Design	4
Die CSS-Regel „display: flex “	4
Der Flex-Container	5
Flex-Direction	8
Verhalten beim Zeilen- bzw. Spaltenende: flex-wrap	11
Flex-Items	14
Die Eigenschaft „order“	14
Die Eigenschaft „flex“	15
flex-basis.....	16
flex-grow.....	16
flex-shrink.....	20
Kombination von flex-base, flex-grow und flex-shrink	21
Ausrichtung der Flex-Item	21
justify-content	21
align-items.....	23
align-self	25
Einsatz-Beispiele der Flex-Technologie	26
Struktur einer Webseite	26
Wirkung von „order“	27
Verwendung der Flex-Technologie in Verbindung mit @media-Queries	29
Ausrichtung eines Menü-Blocks.....	31
Zentrierte Texte in einem Button.....	32
Reflexionsfragen.....	34

Responsives Design

Im vorherigen Modul (@media/@container-Queries) wurden Möglichkeiten aufgezeigt, die Eigenschaften des Ausgabemediums abzufragen und diese mithilfe von CSS für angepasste Formate auszuwerten. Ein echtes responsives Design bedeutet allerdings, dass Elemente der Webseite entsprechend der Darstellungsmöglichkeiten (z.B.: Viewportbreite) automatisch angepasst werden.

Im Modul „@media-Queries“ wurde auch deutlich, dass es keine pauschale Richtlinie geben kann, unter welchen Bedingungen eine Webseite auf einem Smartphone optimal wiedergegeben werden kann. Das liegt in der Vielfalt der Geräte begründet sowie in der Tatsache, dass Browserfenster individuell in ihren Abmessungen auf dem Screen platziert werden können. Es stellt sich also grundsätzlich die Frage, bei welchen Abmessungen die Inhalte der einzelnen Elemente noch gut oder zumindest in angemessener Qualität dargestellt werden können.

Die Auszeichnungssprache HTML bringt eine gewisse Grund-Responsivität mit. So haben (unformatierte) **Block**-Elemente die Breite des Viewports bzw. – im Fall einer Verschachtelung – ihres Eltern-Elements. Wird die maximal verfügbare Breite erreicht, wird die Seite umbrochen, d.h. folgende Elemente werden in eine neue Zeile verschoben. „Überbreite“ Block-Elemente bewirken das Erscheinen einer Scrollbar. Man sieht also im letzteren Fall nur einen Teil des Inhaltes der Seite. Dies ist nicht wirklich wünschenswert.

Die nachfolgend beschriebene Technik soll die Position der Elemente an die Abmessungen des Viewports anpassen, wobei es nicht erforderlich ist, Reihenfolge oder Inhalt der Webseite im HTML-Code zu verändern. Eine sauber programmierte Webseite (HTML) kann also sehr einfach mithilfe der „Flex“-Technologie responsiv auf verschiedenen Ausgabegeräten dargestellt werden. Die (im Idealfall sehr wenigen) Anpassungen erfolgen ausschließlich im CSS-Code innerhalb von @media-Containern.

Die CSS-Regel „display: flex“

Mit der CSS-Eigenschaft „**display**“ lässt sich - wie bereits im Modul 1 zu sehen war – festlegen, ob ein Element als Block-Element (mit Zeilenumbruch) oder als Inline-Element – ohne Zeilenumbruch, im Textfluss integriert – erscheinen soll. Mit *Display* kann auch eine „Mischform“ vorgesehen werden, bei der ein Element nach außen wie ein Inline-Element wirkt, sich also in den regulären Textfluss wie ein einzelner Buchstabe integriert, nach innen jedoch als Block-Element formatiert werden kann. Dies haben Sie beispielsweise bei der Gestaltung waagerecht ausgerichteter Navigationsleisten kennengelernt (Beispiel: `nav li {display: inline-block;}`).

Die Eigenschaft „*display*“ ist auch von der Möglichkeit bekannt, gezielt einzelne Elemente einzublenden, während andere Elemente versteckt bleiben. Zur Erinnerung:

```
display: none;
```

bewirkt das Ausblenden¹ von Elementen. Beispielsweise lassen sich auf diese Weise alle Elemente vom Typ <article> komplett ausblenden. Damit die Seite nicht leer bleibt und die Arbeit bei der Verfassung der <article>-Inhalte nicht umsonst ist, wird mithilfe der dynamischen Pseudoklasse *:target* ein einziges – im Menü gewähltes und über eine ID verlinktes – <article>-Element eingeblendet.

Diese Verwendung der CSS-Eigenschaft „*display*“ ist also bereits bekannt.

¹ Alternativ zu „display: none;“ kann auch die CSS-Eigenschaft „*visibility: hidden;*“ dazu verwendet werden, ein Element „unsichtbar“ zu machen. Allerdings reserviert „visibility“ den für das Element ursprünglich erforderlichen Platz auf der Webseite, was dazu führen kann, dass große freie Flächen erscheinen. Dies lässt sich zwar durch zusätzliche Formate in Höhe und Breite (beides auf Null setzen) ausgleichen, erfordert jedoch zusätzlichen Programmieraufwand.

Die CSS-Eigenschaft „display“ kann jedoch noch viel mehr bewirken. So ist es möglich, mithilfe von „display“ – in Verbindung mit weiteren – nach genauen Regeln aufeinander abgestimmten – CSS-Eigenschaften eine Seite vollkommen neu zu strukturieren:

- Mehrspaltige Anordnung der Elemente in der Seite
- Änderung der Reihenfolge von Elementen in der Seite
- Allgemeine Festlegung von Abständen zwischen den Elementen
- Allgemeine Festlegung zur Ausrichtung der Elemente

Die CSS-Eigenschaft „display“ wird an dieser Stelle verwendet, um als Container für die Darstellung der darin eingebundenen Elemente zu wirken. Neben dem – im folgenden Modul erläuterten – „Grid-Layout“ sollen in diesem Modul die Einsatzmöglichkeiten des „Flex-Layouts“ beschrieben werden.

Hinweis: In Kombination mit den CSS-Medienabfragen (@media und @container) lassen sich mit dem Flex-Layout sehr einfach und effektiv responsive Webseiten umsetzen, welche ohne Änderungen im HTML-Code auf allen Ausgabegeräten funktionieren.

Der Flex-Container

Ein „Flex-Container“, welcher mit der CSS-Regel „display: flex;“ als solcher deklariert und mit weiteren, in diesem Abschnitt vorgestellten Eigenschaften in seiner Wirkung beschrieben wird, enthält die sogenannten Flex-Item (s.u.). Die Flex-Item sind die „Kind-Elemente“ innerhalb des Flex-Containers. Bei ihnen handelt es sich um HTML-Elemente, welche nach den Formaten des Flex-Layouts in der Seite angeordnet werden. Bei den Regeln dieser Anordnung kann es sich u.a. um

- deren Breite / Höhe sowie um
- deren Reihenfolge handeln.

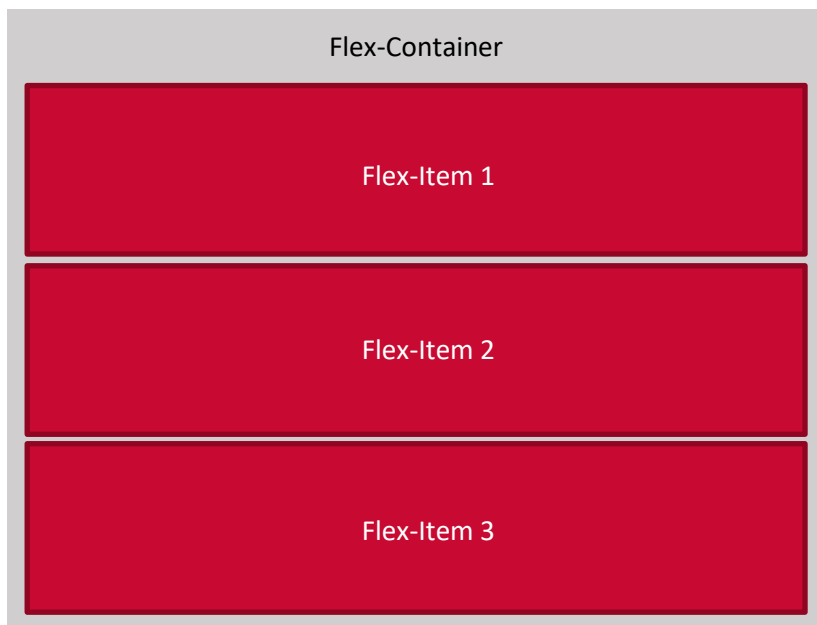


Abbildung 1: Beispiel für ein einfaches Flex-Layout: Der Flex-Container enthält hier drei HTML-Elemente, welche die Flex-Item darstellen. Nur die Kind-Elemente, also Elemente, die direkt in der Hierarchie als erste Ebene innerhalb des Flex-Containers platziert sind, sind Flex-Item.

Es sei zunächst der **Flex-Container** betrachtet. Dieser wird aus einem – die späteren Flex-Item umschließenden – HTML-Element gebildet. Theoretisch kann bereits das Element <body> zu einem Flex-Container deklariert werden. Alle unmittelbaren „Kind-Elemente“ (z.B.: <header>, <nav>, <main>, <footer> etc.) sind dann die Flex-Item. Die Flex-Technologie kann auch zur Gestaltung von Menüleisten

verwendet werden. Hier wird dann zum Beispiel das <nav>- oder das -Element zum Flex-Container deklariert und dessen Inhalte als Flex-Item betrachtet.

Hinweis: Gängige Recherche- und Schulungsportale erklären das Flex-Layout fast ausschließlich mit <div>-Elementen, denen zur Unterscheidung Klassen-Attribute zugewiesen werden. Grund: Die Schulungsportale formulieren **ALLGEMEINE** Beispiele. In Ihren Ausbildungsmodulen verwenden Sie jedoch zunächst eine vorgegebene Architektur, um ein allgemeines Verständnis zu schaffen. Bitte Arbeiten Sie in Ihrer Ausbildung **AUSSCHLIESSLICH** mit den semantischen HTML5-Elementen.

Die bisweilen in den Trainings-Webseiten verwendete HTML-Struktur sieht wie folgt aus:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Flex1</title>
</head>
<body>
  <header>
...
  </header>
  <nav>
...
  </nav>
  <main>
    <article id="art1">
      <p>Artikel 1</p>
    </article>
    <article id="art2">
      <p>Artikel 2</p>
    </article>
    <article id="art3">
      <p>Artikel 3</p>
    </article>
    <article id="art4">
      <p>Artikel 4</p>
    </article>
  </main>
  <footer>
...
  </footer>
</body>
</html>
```

Diese Struktur wird als Grundlage der folgenden Erklärungen dienen. Den Elementen sind bisweilen nur einfache Inhalte (jeweils ein Absatz-Element <p>) zugewiesen worden. Damit sie im Webbrowser besser

sichtbar werden, werden ihnen mit dem nachfolgenden CSS-Code individuelle Hintergrundfarben sowie eine Höhe und Breite zugewiesen.

Tipp: Für die folgenden Experimente können diese beiden Codes in eine Musterdatei gespeichert werden, auf die immer wieder zugegriffen werden kann. Die Flex-Technologie ist hier noch nicht eingebunden! Die Farbgebung ist selbstverständlich wahlfrei. Bei den Höhen und Breiten dürfen Sie gerne etwas experimentieren.

```
main {
  background-color: #faebd7;
}
article {
  background-color: gray;
  height: 100px;
  width: 100px;
}
article:nth-child(1)2 {
  background-color: #b76969;
}
article:nth-child(2) {
  background-color: #89c2e8;
}
article:nth-child(3) {
  background-color: #a5ecb1;
}
```

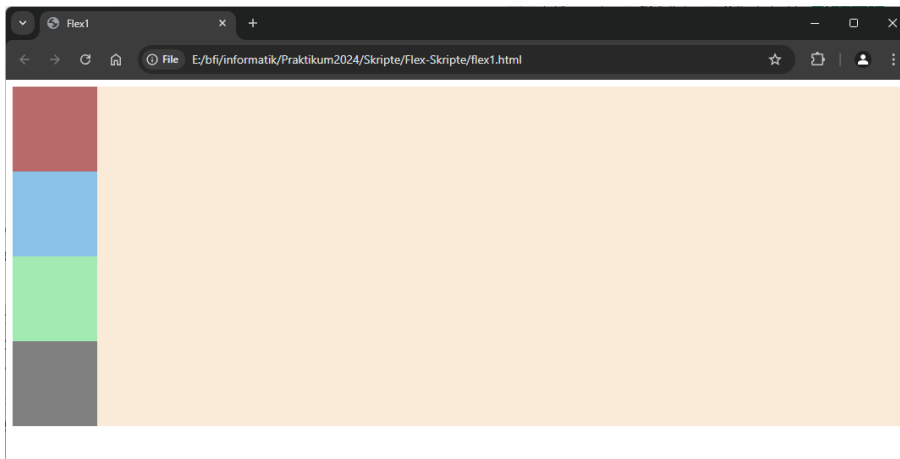


Abbildung 2: Der beschriebene HTML- und CSS-Code bringt ein Ergebnis, wie es im Bild gezeigt wird. Die vertikale Ausrichtung ist durch die Block-Eigenschaft der <article>-Elemente begründet.

Es hängt grundsätzlich vom individuellen Design ab, wie die Flex-Technologie eingesetzt wird. So kann sie zur Ausrichtung und für die Aufteilung von Abständen in Navigationsleisten aber auch – in Verbindung mit den @media-Queries aus dem vorangegangenen Modul – zur Umsetzung responsiver Designs mit CSS eingesetzt werden.

² Die *strukturellen CSS-Pseudoklassen* wie z.B. :nth-child() werden in einem späteren Modul ausführlich behandelt! Vorab kann man sich sehr gut in der Publikation selfhtml.org im Abschnitt CSS – Selektoren informieren.

Vorsicht – Nicht den Überblick verlieren! Die Flex-Technologie wird gerne für alle möglichen Verteilungen von Elementen in einer Webseite verwendet, z.B.: Menüleiste. Da jedoch ein *responsives Design* – Aufteilung und Platzierung in der gewünschten Reihenfolge von Elementen, abhängig von Viewport-Breiten bzw.-Höhen – im modernen Webdesign wichtig ist, ist die Flex-Technologie **EINE** Möglichkeit, Webseiten an die verschiedenen Ausgabegeräte anzupassen. Das wird der Schwerpunkt dieses Moduls sein.

Flex-Direction

Das Flex-Layout sieht ohne weitere Festlegung eine horizontale Anordnung der im Flex-Container enthaltenen Kind-Elemente³ (Flex-Item) von links nach rechts vor. Die horizontale Anordnung der Flex-Item erfolgt entlang der sogenannten **Hauptachse**.

Möglich ist auch eine vertikale Anordnung der Flex-Item entlang der sogenannten **Querachse** (von oben nach unten). Beide Anordnungskonzepte (Hauptachse oder Querachse) lassen sich auch in die entgegengesetzten Richtungen (von rechts nach links auf der Hauptachse sowie von unten nach oben auf der Querachse) umsetzen.

Der ab Seite 6 vorgestellte HTML-Code zeigt vier <article>-Elemente, die direkt in einem <main>-Container eingebunden sind. Sie sind also „Kind-Elemente“ des <main>-Containers. Der Container <main> soll nun als **Flex-Container** dienen. Dazu wird der CSS-Code (ab Seite 7) etwas ergänzt:

```
main {  
    background-color: #faebd7;  
    display: flex;  
}
```

Die rot hervorgehobene CSS-Regel deklariert das Element <main> als **Flex-Container**. Das hat sofort Konsequenzen, wie es in Abbildung 3 gezeigt wird. Das gleiche Ergebnis kann man auch durch Ergänzung einer zusätzlichen CSS-Zeile erreichen:

```
main {  
    background-color: #faebd7;  
    display: flex;  
    flex-direction: row;  
}
```

Hinweis: Default-Wert für die Ausrichtung der Flex-Item innerhalb eines Flex-Containers ist „**flex-direction: row;**“ – Diese Zeile kann also theoretisch im CSS-Code entfallen.

„**Row**“ ist der englische Begriff für eine Zeile. Dementsprechend sind die Ausrichtungen „row“ und – wie gleich zu sehen sein wird – „row-reverse“ immer Ausrichtungen, die sich auf die Hauptachse beziehen.

³ Im Webdesign werden immer wieder „familiäre“ Begriffe wie „Eltern“ (Parent), „Kind“ (Child) oder allgemein „Nachfahren“ verwendet. Direkt in einem Container eingefasste Elemente bezeichnet man als „Kind-Element“ (Child) des Containers, welcher sinngemäß der „Eltern-Container“ (Parent) ist. Elemente, die sich innerhalb eines solchen „Kind-Elements“ befinden, werden als Nachfahren bezeichnet. Sie folgen also nicht direkt der oberen Hierarchie des obersten Containers.

Zu merken: <eltern> <kind> <nachfahre></nachfahre> </kind> </eltern>

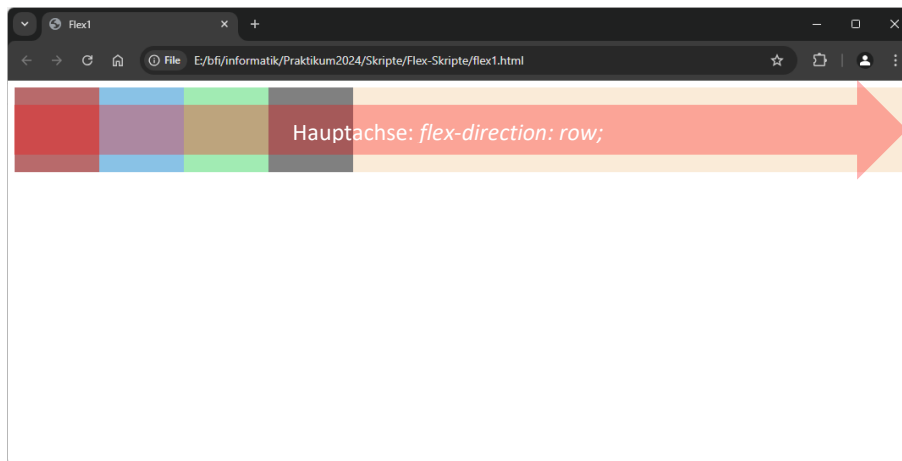


Abbildung 3: Der die vier `<article>` umschließende `<main>`-Container wurde als Flex-Container deklariert. Die Anordnung der ursprünglich untereinander platzierten (Block-)Elemente verändert sich. Sie "fließen" nun horizontal von links nach rechts. Dies entspricht der CSS-Regel „`flex-direction: row;`“

Entsprechend kann die Ausrichtung nach wie vor auf der Hauptachse auch in entgegengesetzter Richtung festgelegt werden. Die dazu erforderliche CSS-Eigenschaft im Regelblock des Flex-Containers lautet: „`flex-direction: row-reverse;`“. Der CSS-Code wird nun in der rot betonten Zeile wie folgt verändert:

```
main {
  background-color: #faebd7;
  display: flex;
  flex-direction: row-reverse;
}
```

Hinweis: Der Begriff „reverse“ bedeutet so viel wie rückseitig oder entgegengesetzt. Im Zusammenhang mit dem CSS-Flex-Format bedeutet es entgegengesetzt zur normalen Schreib- bzw. Leserichtung. In Mitteleuropa liest man von links nach rechts (waagerechte Ausrichtung) und die „reverse Ausrichtung“ (row-reverse) ist dementsprechend von rechts nach links.

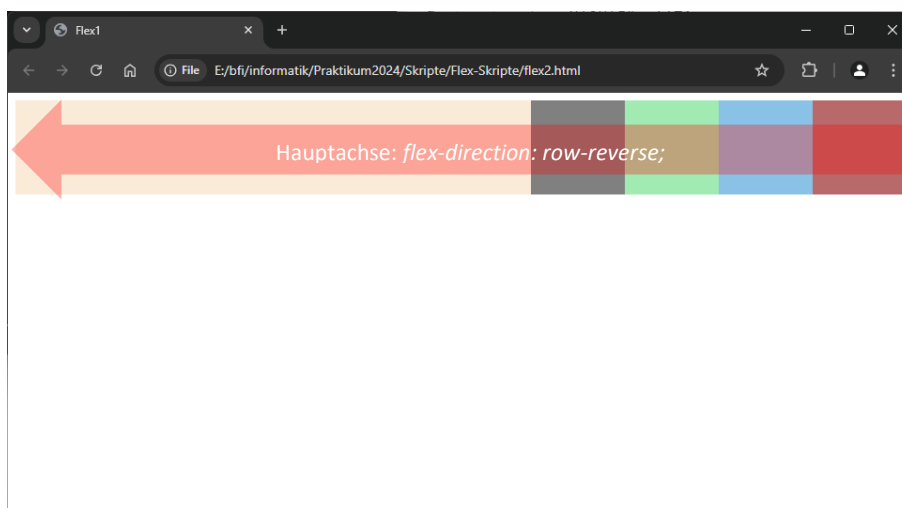


Abbildung 4: Immer noch der gleiche HTML-Code, jedoch hat sich im CSS-Code eine Zeile verändert: Die Flex-Direction wird hier mit „`row-reverse`“ deklariert. Die Ausrichtung bleibt horizontal, jedoch werden die Elemente von rechts nach links angeordnet.

Neben der Anordnung in Bezug zur Hauptachse können die Flex-Item auch senkrecht, also bezogen auf die Querachse ausgerichtet werden. Hier leitet sich der Wert für „flex-direction“ vom englischen Wort für Spalte – column – ab.

```
main {  
  background-color: #faebd7;  
  display: flex;  
  flex-direction: column;  
}
```

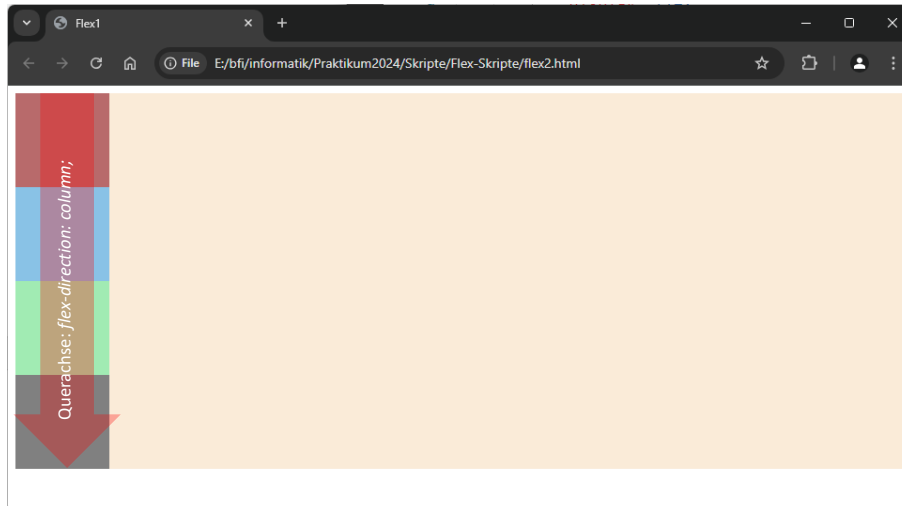


Abbildung 5: Nach wie vor ist der HTML-Code unverändert. Der Wert für die CSS-Eigenschaft "flex-direction" lautet nun jedoch: "flex-direction: column;"

Natürlich kann die Ausrichtung auch umgekehrt (reverse) von unten nach oben erfolgen. Dazu wird die Eigenschaft der flex-direction mit „column-reverse“ festgelegt.

```
main {  
  background-color: #faebd7;  
  display: flex;  
  flex-direction: column-reverse;  
}
```

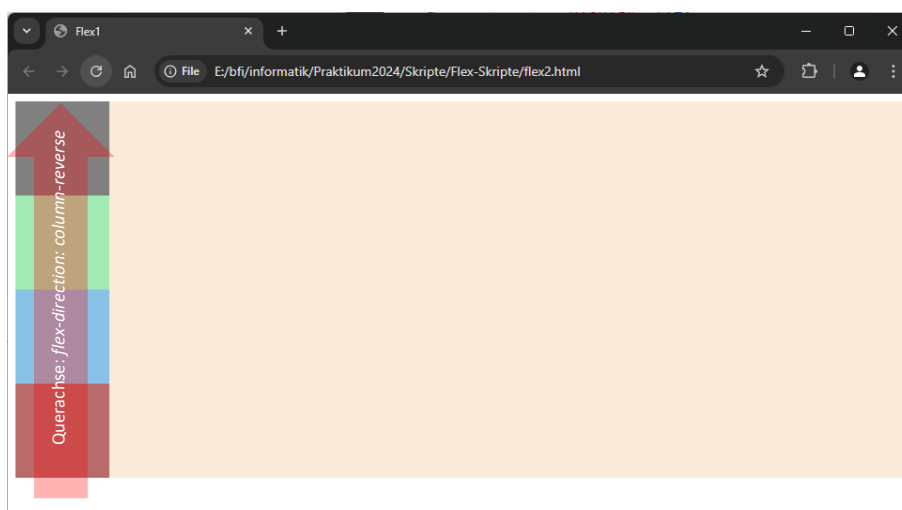


Abbildung 6: Hier wurde - entsprechend den vorherigen Ausführungen in diesem Modul – erneut die CSS-Zeile mit der Deklaration der flex-direction verändert. Der zugewiesene Wert ist nun „column-reverse“.

Hinweis: Immer wieder ist es ein „beliebter“ Schreibfehler: **column** muss mit „mn“ am Ende geschrieben werden.

Verhalten beim Zeilen- bzw. Spaltenende: flex-wrap

Wunderschön! – Sie können nun Elemente mithilfe der Flex-Technologie horizontal oder senkrecht von links nach rechts oder von unten nach oben ausrichten. Es stellt sich aber nun die Frage, was wohl passiert, wenn der Viewport zu klein, also zu schmal oder zu niedrig ist. Als Beispiel für die folgenden Erklärungen soll eine waagerechte Ausrichtung von links nach rechts angenommen werden. Der HTML-Code wird – um den Effekt besser zu veranschaulichen – von vier auf acht `<article>`-Elemente erweitert werden.



Abbildung 7: Der HTML-Code für diese Darstellung entspricht dem der vorangegangenen Beispiele. Folgende Änderungen wurden jedoch vorgenommen: Statt vier gibt es nun acht `<article>`-Elemente. Die Farben wurden durch eine Ergänzung des CSS-Codes frei vergeben. Nach wie vor enthalten die `<article>`-Elemente nun einfache Absatz-Elemente `<p>` mit einer Bezeichnung (z.B.: Artikel 1).

Anmerkung: Auf die Darstellung des neuen HTML- und CSS-Codes wird an dieser Stelle verzichtet. Es wird in diesem Modul vorausgesetzt, dass Sie die neuen Elemente ohne Schwierigkeiten selbst ergänzen können.

Es soll nun der Viewport schmaler werden und zwar so weit, dass die `<article>`-Elemente nicht mehr in ihrer ursprünglich festgelegten Breite (es wurde im Regelblock mit dem Typenselektor `article` die Regel `width: 100px;` eingetragen) in das Fenster hineinpassen. Zu beobachten ist ein Effekt, wie er in Abbildung 8 gezeigt wird. Die Inhalte des Flex-Containers werden quasi zusammengepresst. Das ist optisch nicht attraktiv. Es wird eine weitere CSS-Flex-Eigenschaft im Flex-Container verwendet: „flex-wrap“.

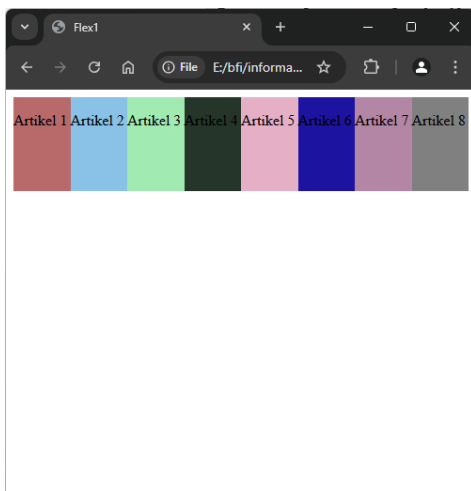


Abbildung 8: Der Viewport wurde stark verschmälert. Durch die Deklaration als Flex-Container (ohne weitere Zusatzformate) werden aber auch die Inhalte des `<main>`-Elements zusammengestaucht. Dies ist nicht unbedingt erwünscht!

Es wird die im folgenden CSS-Code rot markierte Zeile eingefügt:

```
main {
  background-color: #faebd7;
  display: flex;
  flex-direction: row;
  flex-wrap: wrap;
}
```

Die CSS-Regel „flex-wrap: wrap;“ (wrap bedeutet im Englischen „umwickeln“) bewirkt, dass der Inhalt eines Flex-Containers in eine neue Zeile umbrochen wird, wenn die Grenzen des Viewports erreicht sind. Das Ergebnis des vorangehenden CSS-Codes zeigt entsprechend Abbildung 9. Die Elemente behalten ihre ursprünglich festgelegte Größe, verschieben sich aber am Rand des Viewports in eine neue Zeile.

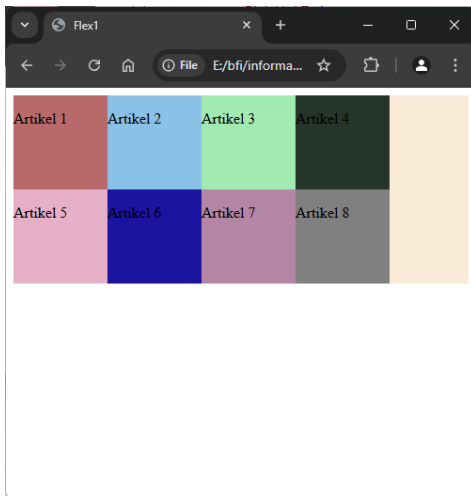


Abbildung 9: Hier wird keines der Flex-Item in seiner Breite verändert. Stattdessen wird durch die CSS-Regel "flex-wrap: wrap;" ein Zeilenumbruch bewirkt.

Selbstverständlich funktioniert „flex-wrap: wrap;“ auch in der reversen Variante der Ausrichtung an der Hauptachse. Wieder soll der CSS-Code ein wenig angepasst werden:

```
main {
  background-color: #faebd7;
  display: flex;
  flex-direction: row-reverse;
  flex-wrap: wrap;
}
```

Beachten Sie nun in Abbildung 10 den Verlauf der Artikel 1 bis 8 und vergleichen Sie dieses Bild mit Abbildung 11!

In Abbildung 10 wirkt die reverse Ausrichtung auf der Hauptachse (CSS_Regel: „flex-direction: row-reverse;“. Es dominiert also die Leserichtung von rechts nach links in einer Zeile (Ausrichtung zur Hauptachse). In Abbildung 11 wurde dagegen die Leserichtung (row, also entlang der Hauptachse von links nach rechts) beibehalten, allerdings wurde der Umbruch (wrap) invertiert. Die Zeile wird also von unten nach oben umbrochen, wobei aber die Leserichtung von links nach rechts gleichbleibt.

Achtung: Bitte seien Sie vorsichtig, wenn Sie inverse Werte für die CSS-Eigenschaften flex-direction und flex-wrap verwenden.

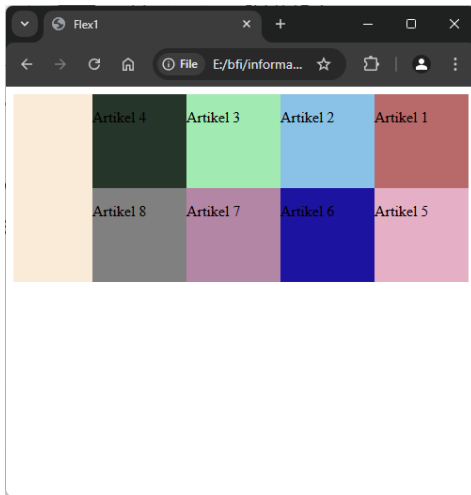


Abbildung 10: Durch die Regel „flex-direction: row-reverse;“ werden die Elemente von rechts nach links, also entgegen der europäischen Leserichtung angeordnet. Die Regel „wrap: wrap;“ erzwingt einen Zeilenumbruch, wenn der verfügbare Platz im Viewport nicht mehr ausreicht, um ein Element in seiner definierten Breite darzustellen.

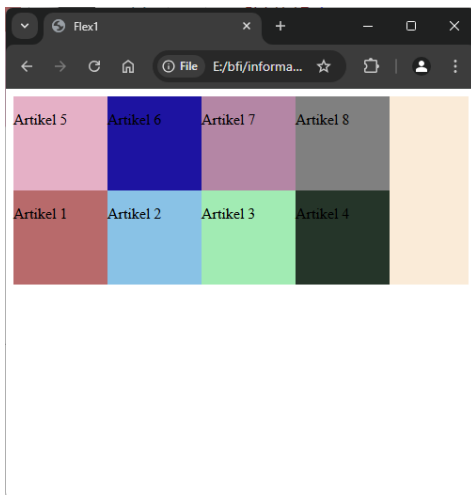


Abbildung 11: Es wurde in diesem Bild die Leserichtung nach dem allgemeinen Standard von links nach rechts festgelegt („flex-direction: row;“). Der Umbruch ist jedoch „reverse“, also von unten nach oben vorgegeben („wrap-reverse;“). Das erste Element erscheint deshalb links unten.

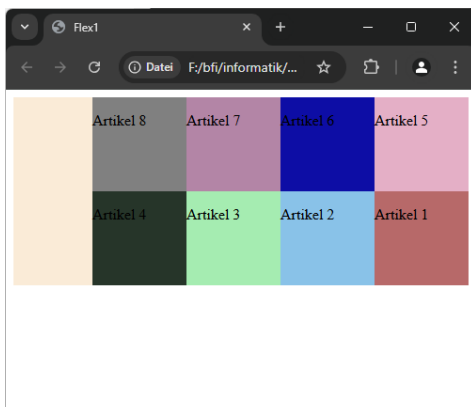


Abbildung 12: Sowohl die Leserichtung („flex-direction: row-reverse;“) als auch der Umbruch sind „reverse“ vorgegeben („wrap: wrap-reverse;“). Das erste Element erscheint hier also unten rechts. Die Leserichtung erfolgt von rechts nach links und der Umbruch von unten nach oben.

Flex-Items

Die Eigenschaft „order“

In den Beispielen ist deutlich zu sehen, dass die farbigen Elemente (hier: <article>) in der Reihenfolge ihrer Programmierung im HTML-Code durchnummeriert⁴ sind. Soll nun die Reihenfolge dieser Elemente verändert werden, so könnte dies natürlich durch entsprechende Verschiebung der Elemente innerhalb des HTML-Codes erfolgen, was jedoch sehr unprofessionell ist und zudem keine Änderung der Reihenfolge im Rahmen eines responsiven Designs zulässt.

Für die bessere Übersicht sollen an dieser Stelle wieder der HTML-Code ab Seite 6 (nur vier <article>-Elemente) und der CSS-Code ab Seite 7 verwendet werden. Das <main>-Element ist wieder der Flex-Container mit den folgenden CSS-Regeln:

```
display: flex;
flex-direction: row;
flex-wrap: wrap;
```

Das Ziel soll es nun sein, die HTML-Elemente innerhalb des Flex-Containers neu in der Reihenfolge zu ordnen, ohne den HTML-Code zu verändern. Das ist bei einem Flex-Container sehr einfach mit einer neuen CSS-Eigenschaft – order – möglich. Abbildung 13 zeigt das ursprüngliche Ergebnis in der Browser-Ansicht. Gleich anschließend ist in Abbildung 14 zu sehen, wie die CSS-Eigenschaft „order“ auf die einzelnen Flex-Item wirkt.

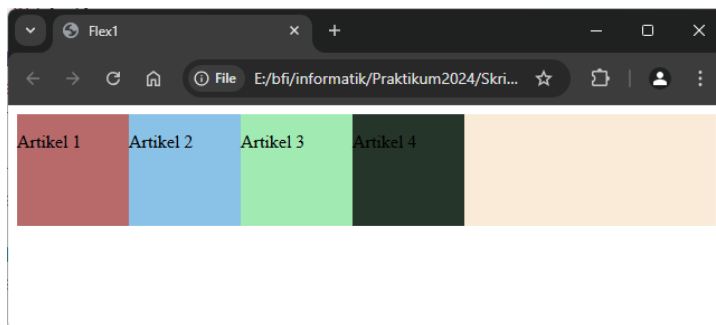


Abbildung 13: Die ersten vier Artikel in der Reihenfolge ihrer Programmierung im HTML-Code: Leserichtung ist von links nach rechts.

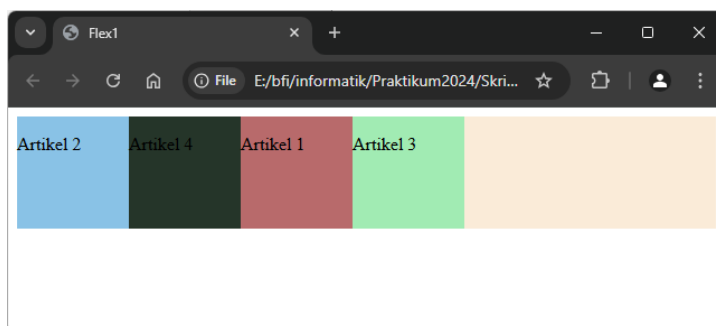


Abbildung 14: Jedem Element wurde im CSS-Code eine „Order-Eigenschaft“ zugewiesen, deren einziger Wert jeweils die Reihenfolge in der Anzeige sein soll.

Anstatt in der Reihenfolge „1, 2, 3, 4“ (Abbildung 13), wie die Elemente im unformatierten HTML-Code programmiert wurden, erscheinen diese nun in Abbildung 14 in der Reihenfolge „2, 4, 1, 3“. Wie bereits ausgeführt: Der HTML-Code ist unverändert.

⁴ Die Nummerierung erfolgt NICHT automatisch. Es handelt sich lediglich um Textzeilen innerhalb des <p>-Elements, welches im <artikel> eingebunden ist. Das Element <article> ist hier das Eltern-Element des jeweiligen Absatzes.

Interessant sind die zusätzlichen CSS-Code-Zeilen, die in den Formaten der „Flex-Item“, also in den Elementen, die unmittelbar Inhalt des Flex-Containers sind, geschrieben wurden:

```
article:nth-child(1) {
    background-color: #b76969;
    order: 3;
}
article:nth-child(2) {
    background-color: #89c2e8;
    order: 1;
}
article:nth-child(3) {
    background-color: #a5ecb1;
    order: 4;
}
article:nth-child(4) {
    background-color: #263529;
    order: 2;
}
```

Die CSS-Eigenschaft „*order*“ legt anhand der programmierten Werte die Reihenfolge der Flex-Item innerhalb des Flex-Containers fest. Auf diese Weise ist es beispielsweise möglich, in einem responsiven Design, Elemente nach ihrer Wichtigkeit für den Betrachter anzuordnen, wenn der Viewport zu schmal wird. So ist es denkbar, dass in einer Seite ein Werbeblock enthalten ist, der auf einem breiten Viewport links neben dem Hauptteil platziert wird. Der Besucher der Seite wird allerdings bei einem schmalen Viewport schnell das Interesse an der Publikation verlieren, wenn diese Werbung auf dem Smartphone vor dem eigentlich interessanten Inhalt präsentiert wird und der Besucher auf der Seite scrollen muss. Mithilfe der CSS-Eigenschaft „*order*“ kann in einem Flex-Layout festgelegt werden, dass die relevanten Inhalte in einer besseren Position erscheinen.

Die Eigenschaft „flex“

Zuerst einmal soll möglichen Missverständnissen vorgebeugt werden: Im Codeblock auf Seite 8 wurde der Begriff „*flex*“ als **Wert** für die CSS-Eigenschaft ***display*** verwendet. Mit der daraus entstehenden CSS_Regel „***display: flex;***“ wird der betreffende Container als „Flex-Container“ für die darin enthaltenen Kind-Elemente, welche fortan auch als *Flex-Item* bezeichnet werden, deklariert.

Achtung: In der CSS-Regel „***display: flex;***“ ist „display“ die CSS-Eigenschaft und „flex“ deren „Wert“!

Darüber hinaus gibt es auch die **CSS-Eigenschaft** „flex“, welche im CSS-Code des *Flex-Containers* platziert wird und die direkte Auswirkungen auch auf die Breite der Flex-Item sowie deren Verhalten bei Veränderungen des Viewports hat. Der Eigenschaft „flex“ können bis zu drei Werte zugewiesen werden:

- flex-basis
- flex-grow
- flex-shrink

Mithilfe dieser Eigenschaften kann ausgehend von einer Basisgröße das Verhalten eines Flex-Item bei Veränderungen der Viewport-Größe festgelegt werden.

flex-basis

Die „flex-basis“ ist die Ausgangsbreite eines Flex-Item innerhalb des Flex-Containers. Als Werte können die gängigen Maßeinheiten für Breitendeklarationen verwendet werden. Beispiele: px, em, %, vw etc.

Im folgenden Beispiel werden die vier <article>-Elemente, welche sich im Flex-Container (<main>) befinden, mit verschiedenen flex-basis-Werten (100px, 200px, 300px und 400px) in ihrer Breite als Basisbreite definiert. Das Ergebnis zeigt Abbildung 15. Alternativ dazu hätte man natürlich auch die jeweiligen Breitenformate mit der CSS-Eigenschaft „width“ festlegen können. Das Ergebnis in der Darstellung wäre zumindest in Abbildung 15 identisch. Allerdings würden die folgenden CSS-Eigenschaften „flex-grow“ und „flex-shrink“ nicht funktionieren.

Der folgende CSS-Code (Auszug mit nur zwei Formaten) zeigt die Formatierung mit der Eigenschaft „flex-basis“ und individuellen Breiten.

```
article:nth-child(1) {
    background-color: #b76969;
    flex-basis: 100px;
}
article:nth-child(2) {
    background-color: #89c2e8;
    flex-basis: 200px;
}
...

```

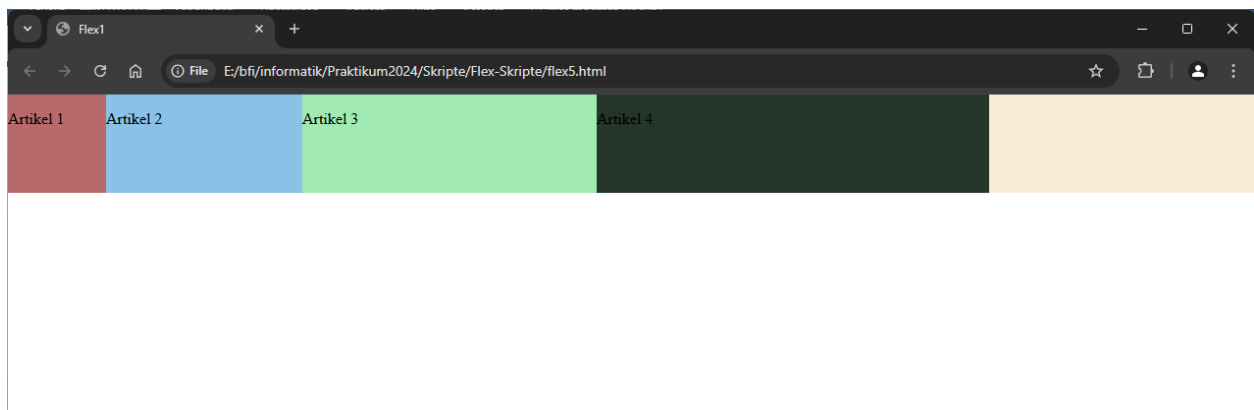


Abbildung 15: Ergebnis des beschriebenen Flex-Layouts mit verschiedenen Werten für flex-basis (100px, 200px, 300px, 400px)

flex-grow

Wenn das Beispiel in Abbildung 15 nachprogrammiert wurde, wird man feststellen, dass sich die Elemente selbst bei einer Veränderung in der Breite des Flex-Containers nicht verändern (verbreitern oder verschmälern). Stattdessen wird, entsprechend der Eigenschaft des Flex-Layouts, möglicherweise ein Zeilenumbruch gesetzt. Nicht mehr vollständig in die Breite der Seite passende Elemente werden in eine neue Zeile⁵ gesetzt, wie es Abbildung 15 zeigt. Das muss nicht immer gewünscht sein.

Hier kann die CSS-Eigenschaft „flex-grow“ (ebenso wie „flex-shrink“) eine Lösung darstellen.

Die CSS-Eigenschaft „flex-grow“ legt das Verhältnis fest, mit dem sich bei einer Vergrößerung des Flex-Containers das betreffende Element in dessen Dimensionen verändert (ebenfalls vergrößert).

Ein Beispiel soll zeigen, wie die Eigenschaft flex-grow wirkt, wie genau sich die Breiten (respektive Höhen bei einem spaltenorientierten Flex-Container) rechnerisch kalkulieren lassen und welchen Einfluss die Eigenschaft auf einen Umbruch (Container ist breiter als die Summe der darin enthaltenen Elemente)

⁵ Bei einer spaltenorientierten Seite entsteht der Umbruch selbstverständlich in eine neue Spalte.

hat. In Abbildung 16 haben alle Artikel-Elemente eine Breite von jeweils 100px. Die Breite des Flex-Containers (Element `<main>`), in welches die Artikel eingeordnet wurden, ist auf 400px justiert⁶. Das *entspricht in diesem Fall* zugleich der Viewport-Breite, weil für den `<main>`-Container keine Breite festgelegt wurde (keine `width`-Eigenschaft). Es ist sehr einfach zu beobachten, dass alle Artikel tatsächlich gleich breit sind, obwohl der CSS-Code, wie er auf Seite 18 zu sehen ist, verschiedene Werte für `flex-grow` vorsieht. Zu beachten sind also die Eigenschaften „**flex-grow**“ in den jeweiligen Containern. Diese sind im gezeigten Code mit roter Schrift hervorgehoben.

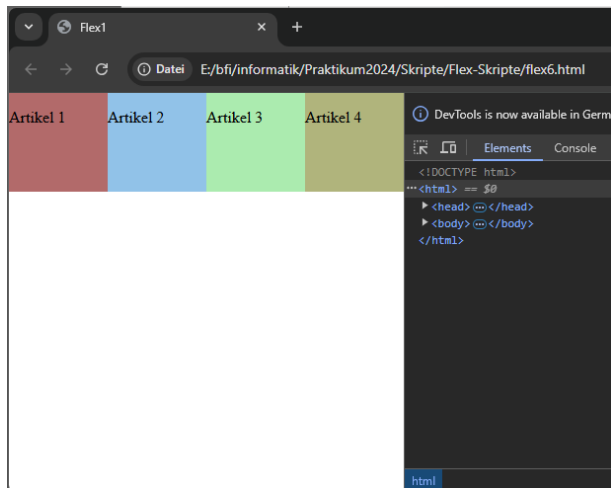


Abbildung 16: Die Breite des Flex-Containers (hier: das Element `<main>`) wurde in diesem Beispiel auf genau 400px (4 x 100px der einzelnen Elemente in der Breite) justiert. Alle Elemente haben im Beispiel genau die gleiche Breite (jeweils 100px).

Hinweis: Wie findet man die Breite eines Elements in einer Webseite heraus? – Hier helfen die Entwicklertools der Webbrowser: Klicken Sie mit der rechten Maustaste in die Webseite und wählen Sie im Kontextmenü *Untersuchen* bzw. *Inspect*. Sie erreichen die Entwicklertools auch über Shortcuts:

- [Umschalt]+[Strg]+[I]: Zuletzt benutztes Entwicklertool
- [Umschalt]+[Strg]+[J]: Konsole
- [Umschalt]+[Strg]+[C]: Elemente (ggfs. kombiniert mit den Formaten)

Die Bereiche der Entwicklertools können über die Reiter gewechselt werden.

Suchen Sie nun das jeweils zu betrachtende Element im Code im Entwickler-Fenster. Es wird in der Webseite das Element markiert und es werden dessen Maße in einem Tooltip angezeigt (vgl. `article#art2` in Abbildung 17).

```
main {
  background-color: #faebd7;
  display: flex;          /* <main> ist der Flex-Container */
  flex-direction: row;
  flex-wrap: wrap;
}
```

⁶ Öffnen Sie die Entwicklertools des Browsers und schieben Sie den sichtbaren Bereich der Seite mit der Maus soweit zusammen bzw. auseinander, bis oben rechts (in der Seite) eine Breite von 400px ablesbar ist.

```

article {
  background-color: gray;
  height: 100px;
  flex-basis: 100px;    /* Das Element <article> hat die Grundbreite 100px */
}
article:nth-child(1) {
  background-color: #b76969;
  flex-grow: 1;
}
article:nth-child(2) {
  background-color: #89c2e8;
  flex-grow: 3;
}
article:nth-child(3) {
  background-color: #a5ecb1;
  flex-grow: 2;
}
article:nth-child(4) {
  background-color: #b1b47d;
  flex-grow: 1;
}

```

Obwohl die Eigenschaft „flex-grow“ mit verschiedenen Werten belegt wurde, sind hier (noch) alle Artikel gleich breit (je 100px). Der Code bleibt für das folgende Experiment absolut unverändert! Nun soll jedoch die Breite des Flex-Containers (hier noch immer das Element <main>) auf 800px eingestellt werden. Es zeigt sich eine vollkommen neue Erscheinung (vgl. Abbildung 17). Die Elemente sind erheblich in ihrer Breite verschieden.

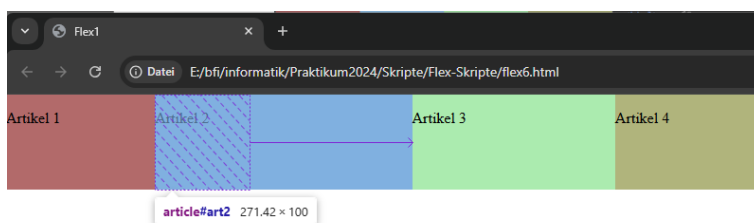


Abbildung 17: Der gleiche Code (sowohl HTML als auch CSS), allerdings wird die **Viewport-Breite** auf 800px eingestellt.

Wird in den Entwicklertools im Bereich „Elemente“ über das zu untersuchende Element gefahren (hier: Artikel 2), so wird symbolisch sichtbar, wie breit das Element in seiner Grundeinstellung (durch flex-basis) ursprünglich war und wie stark sich das Element nun aufgrund der Verdoppelung der Breite des Flex-Containers nun vergrößert hat. Es fällt auf, dass Artikel 2 nicht auf die dreifache Breite gewachsen ist, obwohl die Regel „**flex-grow: 3;**“ gesetzt wurde.

Betrachten wir deswegen die CSS-Eigenschaft „flex-grow“ mathematisch (Achtung: *Bitte etwas Zeit dafür einplanen, um das Prinzip wirklich zu verstehen!*):

Wie gesehen, liegen folgende CSS-Regeln zugrunde (nur die Regeln, die hier tatsächlich relevant sind, werden gelistet):

```
main {... flex-basis: 100px; ...}
article:nth-child(1) {... flex-grow: 1;...}
article:nth-child(2) {... flex-grow: 3;...}
article:nth-child(3) {... flex-grow: 2;...}
article:nth-child(4) {... flex-grow: 1;...}
```

In einer Formel ausgedrückt, sieht die Situation wie folgt aus:

$$100 \cdot (1 + 1x) + 100 \cdot (1 + 3x) + 100 \cdot (1 + 2x) + 100 \cdot (1 + 1x) = 800$$

Grün ist also die Flex-Basis und die **flex-grow**-Werte werden farbig dargestellt. „x“ ist ein hier zu errechnender Faktor, welcher den Grundwert darstellt, um den der durch die Vergrößerung des Viewports gewonnene Platz ausgefüllt wird. Weil die Flex-Basis aber grundsätzlich bestehen bleibt (die Vergrößerung des Elements wird zu dieser addiert), muss in der Klammer „1 + flex-grow * Faktor x“ stehen. Rechts vom Gleichheitszeichen steht hier die aktuelle Breite des Viewports.

Auflösen der Klammern:

$$100 + 100x + 100 + 300x + 100 + 200x + 100 + 100x = 800$$

Zusammenfassung der Werte ohne Faktor x und Zusammenfassung der x-Werte:

$$400 + 700x = 800$$

Noch einmal an dieser Stelle zur Wiederholung: Der rot hervorgehobene Wert „400“ auf der linken Seite der Gleichung entspricht der Summe aller Flex-Basis-Werte im Flex-Container. Der blau hervorgehobene Wert auf der rechten Seite der Gleichung „800“ entspricht der aktuellen Viewport-Breite. Der grün betonte Term „700x“ setzt sich aus dem noch zu errechnenden Faktor für die Vergrößerung und dem Produkt der flex-grow-Summe mit der Flex-Basis zusammen. Den Wert 400 können wir durch Subtraktion auf beiden Seiten der Gleichung schon einmal aus dem Spiel nehmen.

$$700x = 400$$

„x“ soll allein auf der linken Seite stehen. Wir teilen beide Seiten der Gleichung durch 700.

$$x = \frac{400}{700}$$

Ermittlung des Wertes für x (hier beide Seiten geteilt durch 700 => Ziel links: 1x):

$$x = 0,57142$$

Denksportaufgabe:

Mathematisch Begabte werden möglicherweise eine universelle Formel für verschiedene flex-grow-Summen und beliebige Viewport-Breiten ableiten können.

Der errechnete x-Wert gilt für das hier verwendete Beispiel mit einer Breite des Flex-Containers von 800px. Der jeweils errechnete **x-Wert** ist der Faktor, der mit dem Wert zu multiplizieren ist, welcher für das jeweilige Flex-Item mit „flex-grow“ festgelegt wird. Vergleichen Sie die Werte der Tabelle mit denen, die Sie in der Webseite beobachten.

Basisbreite	x-Faktor	Flex-grow-Wert	Endgültige Elementbreite
100px	0,57142	1	157,142
100px	0,57142	3	271,426
100px	0,57142	2	214,284
100px	0,57142	1	157,142
Gesamtbreite (Breite des Flex-Containers)			799,994px (~ 800px)

flex-shrink

Im vorherigen Abschnitt wurde gezeigt, wie die Inhalte des Flex-Containers – also die Flex-Item – auf eine Vergrößerung des Containers reagieren, wenn für jedes Flex-Item ein spezieller Faktor (flex-grow) deklariert wird. Das Verständnis dieses Abschnittes wird im Folgenden vorausgesetzt.

Der CSS-Code wird ein wenig verändert. Von Bedeutung sind die farbigen Zeilen. Eine flex-wrap-Formatierung sollte nicht vorgenommen werden.

```
main {
  background-color: #faebd7;
  width: 100vw;
  display: flex;
  flex-direction: row;
  flex-wrap: wrap; /* Achtung: Kein flex-wrap verwenden!! */
}
article {
  background-color: gray;
  height: 100px;
  flex-basis: 100px;
  flex-grow: 1;
}
article:nth-child(1) {
  background-color: #b76969;
  flex-shrink: 1;
}
article:nth-child(2) {
  background-color: #89c2e8;
  flex-shrink: 3;
}
article:nth-child(3) {
  background-color: #a5ecb1;
  flex-shrink: 2;
}
article:nth-child(4) {
  background-color: #b1b47d;
  flex-shrink: 1;
}
```

Hinweis: flex-wrap versus flex-shrink

Die CSS-Eigenschaft flex-shrink wirkt, wenn die Flex-Item durch Verkleinerung des Flex-Containers ebenfalls in ihrer Breite angepasst werden. Wird jedoch in den Formaten des Flex-Containers die Eigenschaft flex-wrap verwendet, dann werden Flex-Item, welche nicht mehr in die verfügbare Breite des Containers passen, einfach in eine neue Zeile umbrochen. Flex-shrink ist dann quasi wirkungslos.

Die Kalkulation der Element-Breiten der Flex-Item bei Verkleinerung unter Einwirkung von flex-shrink erfolgt ähnlich wie bei flex-grow. Allerdings wird innerhalb der Klammer eine Subtraktion ausgeführt, denn schließlich sollen auch die Flex-Item schmaler werden.

$$100 \cdot (1 - 1x) + 100 \cdot (1 - 3x) + 100 \cdot (1 - 2x) + 100 \cdot (1 - 1x) = 270$$

$$100 \cdot -100x + 100 - 300x + 100 - 200x + 100 - 100x = 270$$

$$400 - 700x = 270$$

$$-700x = -130$$

$$x = \frac{-130}{-700}$$

$$x = 0,185714$$

Basisbreite	x-Faktor	Flex-shrink-Wert	Endgültige Elementbreite
100px	0,185714	1	81,42
100px	0,185714	3	44,29
100px	0,185714	2	62,86
100px	0,185714	1	81,42
Gesamtbreite (Breite des Flex-Containers)			269,99px (~ 270)

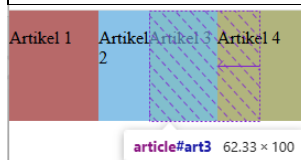


Abbildung 18: Der Viewport hat nunmehr eine Breite von 270px. Durch die CSS-Eigenschaft `flex-shrink`, die für jedes Flex-Item sehr unterschiedlich definiert wurde, sind die Artikel-Elemente auch unterschiedlich stark verkleinert worden.

Kombination von `flex-base`, `flex-grow` und `flex-shrink`

Die CSS-Eigenschaft „flex“ bietet die Möglichkeit, alle zuvor beschriebenen Formate in nur einer einzigen CSS-Zeile zu deklarieren. Die folgende Zeile (bitte die Reihenfolge beachten: `grow – shrink – basis`)

```
flex: 1 3 100px;
```

entspricht den CSS-Formaten:

```
flex-grow: 1;
```

```
flex-shrink: 3;
```

```
flex-basis: 100px;
```

Hinweis: Alle drei Eigenschaften (`flex-grow`, `flex-shrink` und `flex-basis`) können in der CSS-Eigenschaft „flex“ gemeinsam deklariert werden (Bitte Leerzeichen zwischen den Werten nicht vergessen!).

Ausrichtung der Flex-Item

Die Flex-Technologie ermöglicht es, in Verbindung mit den `@media-Queries` gezielt auf Veränderungen in den Abmessungen des Viewports zu reagieren. Eine Webseite kann so sowohl auf einem Smartphone als auch auf einem Monitor eines Desktop-Computers in einer zum Gerät passenden, optimalen Struktur dargestellt werden.

Zudem ist es möglich, die Flex-Item innerhalb eines Flex-Containers nach bestimmten Kriterien an der (horizontalen) Haupt- bzw. Main- und an der (vertikalen) Quer- bzw. Cross-Achse auszurichten.

Die Eigenschaft zur Ausrichtung der Flex-Item innerhalb eines Flex-Containers zur Hauptachse heißt `justify-content`. Die Eigenschaft für die Ausrichtung der Flex-Item an der Querachse ist `align-items`.

`justify-content`

Die Eigenschaft `justify-content` wird in den Regeln für den Flex-Container deklariert und gilt für die horizontale Ausrichtung aller Flex-Item innerhalb dieses Containers.

Für die Illustration wurde der folgende CSS-Code verwendet. Der dazugehörige HTML-Code entspricht den Beispielen dieses Moduls mit vier Flex-Items (vier <article>-Elemente innerhalb des <main>-Elements).

```
main {
  background-color: #faebd7;
  width: 100vw;
  height: 300px;
  display: flex;
  flex-direction: row;
  justify-content: center;
}

article {
  background-color: gray;
  height: 100px;
  flex-basis: 100px;
}

article:nth-child(1) {
  background-color: #b76969;
}

...

article:nth-child(4) {
  background-color: #b1b47d;
}
```

Es können folgende Werte zugewiesen werden:

- **center** Die Flex-Item werden horizontal zentriert auf der Hauptachse ausgerichtet. Die Elemente verteilen sich also gleichmäßig rechts und links der Querachse.
- **flex-start** Die Flex-Item werden *links*bündig auf der Hauptachse ausgerichtet.
- **flex-end** Die Flex-Item werden *rechts*bündig auf der Hauptachse ausgerichtet
- **space-between** Die Flex-Item werden mit gleichmäßigen Abständen zueinander der Hauptachse entlang ausgerichtet. Allerdings wird das erste Flex-Item direkt an den linken und das letzte Flex-Item direkt am rechten Rand des Flex-Containers ohne Abstand platziert.
- **space-around** Der Abstand zwischen den Flex-Item ist grundsätzlich identisch. Es gibt zudem jeweils einen Abstand zum linken und rechten Rand des Flex-Containers, der jedoch nur halb so groß ist wie der Abstand zwischen den Flex-Item.
- **space-evenly** Der Wert „space-evenly“ wirkt ähnlich wie „space-around“, allerdings sind die Abstände zum linken und rechten Rand des Flex-Containers genauso breit wie zwischen den einzelnen Flex-Item.

Die folgenden Abbildungen zeigen die Ergebnisse, wenn in der rot betonten Zeile im vorangegangenen Code-Beispiel die Werte für *justify-content* entsprechend verändert werden.

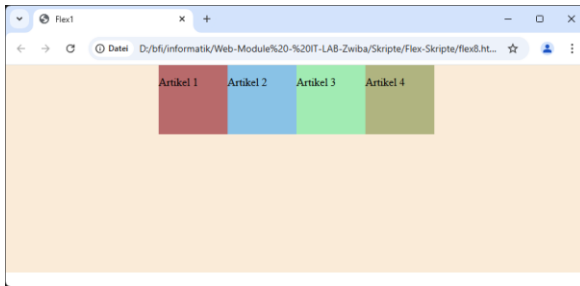


Abbildung 19: Ausrichtung entlang der Hauptachse mit **justify-content: center;**

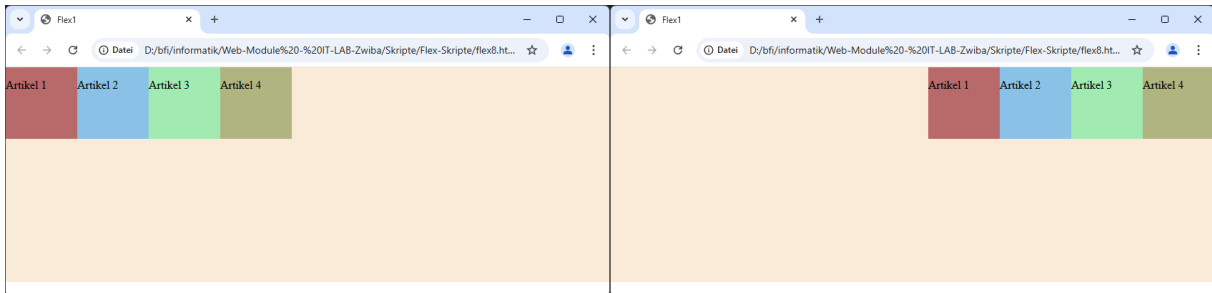


Abbildung 20: Ausrichtung entlang der Hauptachse mit **justify-content: flex-start;** (linkes Bild) und **justify-content: flex-end;** (rechtes Bild)

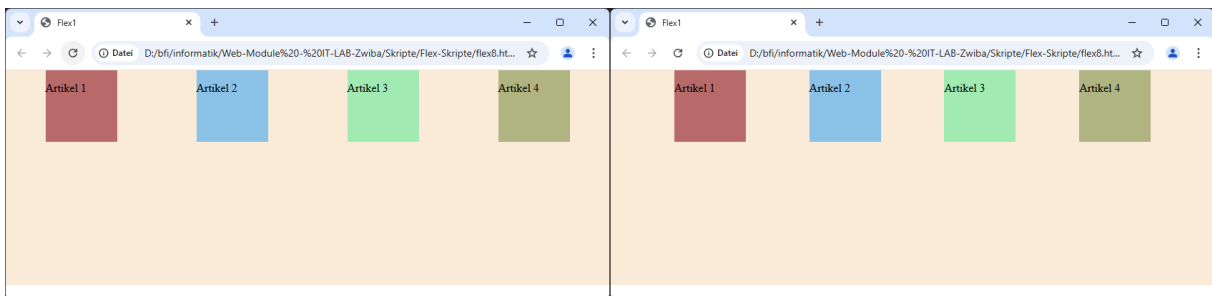


Abbildung 21: Unterschied zwischen **justify-content: space-around;** (linkes Bild) und **justify-content: space-evenly;** (rechtes Bild). In beiden Fällen ist der Abstand zwischen den Flex-Item immer identisch. Im rechten Bild entspricht aber auch der Abstand zu den Rändern dem zwischen den Flex-Item. Bei „**space-around**“ ist dies nicht der Fall.

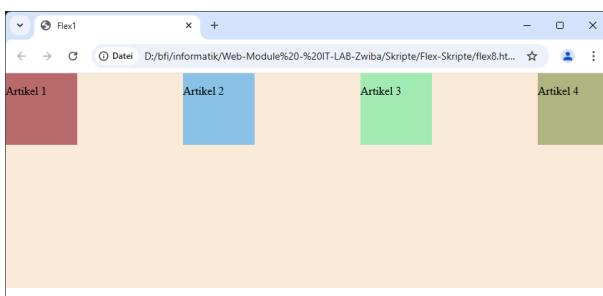


Abbildung 22: Anders als in den beiden vorherigen Bildern zu sehen, gibt es bei der Regel **justify-content: space-between;** keine Abstände zum rechten oder linken Rand des Containers. Die Abstände zwischen den Flex-Item sind jedoch identisch.

align-items

Es soll wieder der CSS-Code von Seite 22 verwendet werden. Die rot markierte Zeile wird jedoch durch die CSS-Eigenschaft „align-items“ mit einem der folgenden Werte ersetzt. Hierbei handelt es sich um die Ausrichtung entlang der (vertikalen) Querachse.

- **center** Die Flex-Item werden vertikal zentriert an der Querachse ausgerichtet. Die Elemente verteilen sich also gleichmäßig ober und unterhalb der (horizontalen) Hauptachse. Der Effekt ist somit eine „vertikale Zentrierung“.

- **flex-start** Mit der Regel „*align-items: flex-start;*“ im Format des Flex-Containers werden die Flex-Item an der Oberkante des Containers und bei einem Umbruch (wrap) nach unten ausgerichtet.
- **flex-end** Mit der Regel „*align-items: flex-end;*“ im Format des Flex-Containers werden die Flex-Item an der Unterkante des Containers und bei einem Umbruch (wrap) nach oben ausgerichtet.
- **stretch** Mit der Regel „*align-items: stretch;*“ füllen die Flex-Item die verfügbare Höhe innerhalb des Flex-Containers aus. Hier ist jedoch unbedingt zu beachten, dass den als Flex-Item definierten Elementen keine Höhe zugewiesen wurde, da diese sonst gegenüber „*align-items*“ dominant ist.
- **baseline** Mit der Regel „*align-items: baseline;*“ nehmen die Flex-Item die Höhe ein, die sie zur Darstellung ihrer Inhalte benötigen. Referenz ist die Grundlinie (baseline) der letzten Inhaltszeile. Dies kann auch über die deklarierte Höhe des Flex-Containers hinaus reichen. Auch hier gilt, dass den Elementen, welche die Flex-Item darstellen, in deren Formatierung keine Höhe zugewiesen werden darf, weil diese sonst gegenüber der *align-items*-Eigenschaft im Flex-Container dominant ist.
- **normal** Default-Wert, wenn keine Regel für *align-items* deklariert wird. Das Verhalten entspricht „*align-items: stretch;*“ mit den dazu bereits erläuterten Einschränkungen.

Hinweis: Für die Werte *baseline* und *stretch* darf keine Höhe für das Flex-Item definiert sein, weil die Höhen-Deklaration für die jeweiligen Elemente dominiert und den Effekt von *align-items* unterdrückt.

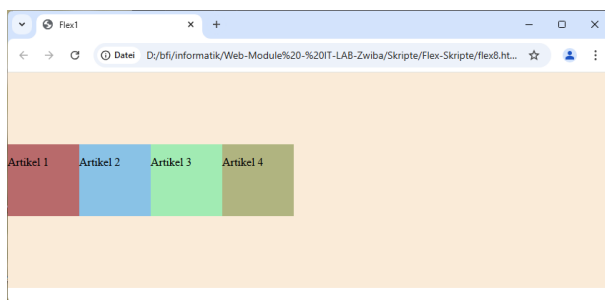


Abbildung 23: Wird lediglich die Regel „*align-items: center;*“ in das Format des Flex-Containers geschrieben, werden die Elemente vertikal zentriert ausgerichtet.

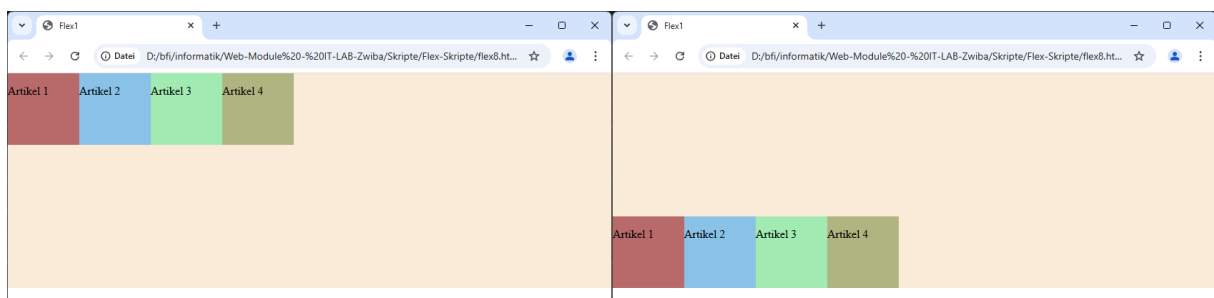


Abbildung 24: Mit der Regel „*align-items: flex-start;*“ werden die Elemente vertikal an der oberen Kante des Flex-Containers ausgerichtet. Ein eventueller Umbruch erfolgt nach unten. Mit der Regel „*align-items: flex-end;*“ ist es dagegen genau umgekehrt: Die vertikale Ausrichtung erfolgt von der Basislinie des Flex-Containers ausgehend nach oben.



Abbildung 25: Die Regel „align-items: stretch;“ bewirkt, dass die Höhe der Flex-Item die volle Höhe des Flex-Containers ausfüllt. Es darf keine fixe Höhe in den Formaten der jeweiligen Flex-Item selbst festgelegt sein!

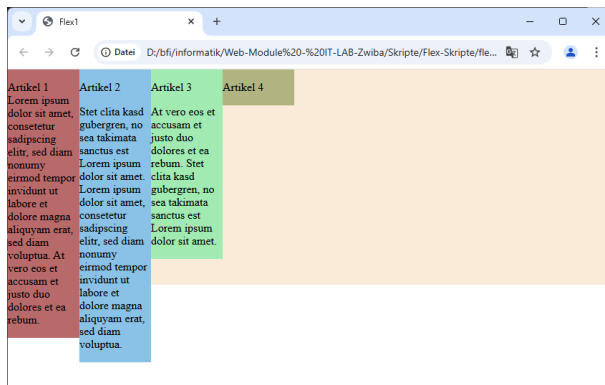


Abbildung 26: Die Regel "align-items: baseline;" definiert die Höhe der jeweiligen Flex-Item anhand derer Inhalte. Es darf keine fixe Höhe in den Formaten der jeweiligen Flex-Item selbst festgelegt sein!

align-self

Mit der Eigenschaft align-items, welche im Format des Flex-Containers deklariert wird, werden die Ausrichtungen *aller* Flex-Items *innerhalb des Flex-Containers* festgelegt. Mit der Eigenschaft **align-self**, die **lediglich in den betroffenen Flex-Item** (NICHT im Flex-Container) verwendet wird, lassen sich für einzelne Elemente Ausnahmen festlegen.

Ein Beispiel (nur Auszug aus dem gesamten Code):

Der Flex-Container (hier: <main>) wird mit dem folgenden Code (Auszug) formatiert:

```
main {
  display: flex;
  flex-direction: row;
  align-items: flex-start;
}
```

Das Flex-Item mit dem Inhalt „Artikel 2“ soll in seiner Ausrichtung eine Ausnahme bilden und am unteren Rand des Flex-Containers erscheinen. Im CSS-Code-Block dieses Elements ist deswegen die grün hervorgehobene Zeile zu finden.

```
article:nth-child(2) {
  align-self: flex-end;
}
```

Probieren Sie auch andere Ausnahmeformate aus!



Abbildung 27: Das Element mit dem Inhalt "Artikel 2" wurde mit „align-self: flex-end;“ formatiert. Obwohl das Format im Flex-Container eine Ausrichtung nach der Regel "align-items: flex-start;" vorsieht, erscheint es deswegen am unteren Rand des Flex-Containers.

Einsatz-Beispiele der Flex-Technologie

Struktur einer Webseite

In Abbildung 28 bis Abbildung 32 wird eine Webseite gezeigt, welche auf dem folgenden HTML-Code (Auszug aus dem Gesamt-Code) basiert.

```
<html>
<head> ... </head>
<body>
  <header>
...
  </header>
  <nav>
    <ul>
      <li><a href="#art1">Artikel 1</a></li>
      ...
    </ul>
  </nav>
  <main>
    <article id="art1">
      ...
    </article>
... <!-- weitere Artikel --> ...
  </main>
  <footer>
    ...
  </footer>
</body>
</html>
```

Darüber hinaus gelten folgende CSS-Formate (Achtung: Dies ist nur ein Auszug aus dem Code! Dynamische Pseudoklassen, Formate der Schaltflächen und Farbgebungen etc. sind hier nicht im Code aufgeführt).

```
body {  
    display: flex;  
    flex-wrap: wrap;  
    flex-direction: row;  
}
```

Der `<body>` der Seite ist auch gleichzeitig der *Flex-Container*. Das bedeutet, dass seine direkten *Kind-Elemente* (hier: `<header>`, `<nav>`, `<main>` und `<footer>`) die *Flex-Item* sind. Anders sieht es dagegen bei den Elementen `<article>` (ist Inhalt des Containers `<main>`), `<ul>`, `<li>` und `<a>` (sind Inhalte des Containers `<nav>`) sowie natürlich die weiteren Inhaltselemente der Flex-Item aus. Sie sind zwar alle *Nachfahren* des `<body>`-Containers, jedoch keine direkten Kind-Elemente und somit keine Flex-Item.

Das Element `<body>` wurde im ursprünglichen Code mit der Eigenschaft „*display*“ zum Flex-Container ernannt. Die weiteren Eigenschaften sehen einen Umbruch am Rand des Containers und eine horizontale Ausrichtung von links nach rechts vor.

```
header {  
    order: 1;  
    flex: 100%;  
    height: 80px;  
}  
  
nav {  
    order: 2;  
    flex: 24%;  
    margin-right: 5px;  
}  
  
main {  
    order: 3;  
    flex: 75%;  
}  
  
footer {  
    order: 4;  
    flex: 100%;  
}
```

Wirkung von „order“

Die bereits besprochene Eigenschaft „*order*“ (vgl. ab Seite 14) legt die Reihenfolge der Elemente in ihrem Erscheinen in der Webseite fest. Dabei spielt es – wie zu sehen sein wird – keine Rolle, in welcher Reihenfolge die Elemente tatsächlich programmiert wurden. In diesem Fall wird folgende Reihenfolge festgelegt.

1. <header>: Mit der zusätzlichen Eigenschaft „flex: 100%;“ wird dem Element die volle Seitenbreite zugewiesen. Das bedeutet, dass das nachfolgende Element automatisch in der nächsten Zeile⁷ platziert wird. Zudem wird die Höhe des Headers mit 80px festgelegt.
2. <nav>: Diesem Flex-Item wird mit „flex: 24%;“ nur knapp ein Viertel der Breite des Flex-Containers zugewiesen. Da es keine anderweitige Formatierung gibt, erscheint der Navigationsbereich am linken Rand des Containers.
3. <main>: Rechts neben dem <nav>-Container ist noch viel Platz, da dieser nur 24% der Flex-Container-Breite belegen soll. Dem <main>-Container wurde mit „flex: 75%“ ebenfalls nicht die volle Breite einer Zeile zugewiesen. Er passt jedoch sehr gut neben den zuvor platzierten <nav>-Container, so dass hier ein zweispaltiges Layout entsteht.
4. <footer>: Der Footer wird an die vierte Stelle gesetzt auch ihm wird wie beim Header die volle Breite des Flex-Containers zugewiesen (flex: 100%;).

Es entsteht ein Bild, wie es die Abbildung 28 zeigt. Wie bereits angedeutet, sind den Elementen verschiedene Hintergrundfarben zugewiesen worden. Da der Umgang mit den CSS-Grundformaten mittlerweile vorausgesetzt werden kann, sind die dafür erforderlichen Code-Zeilen nicht mehr in diesem Skript aufgeführt.

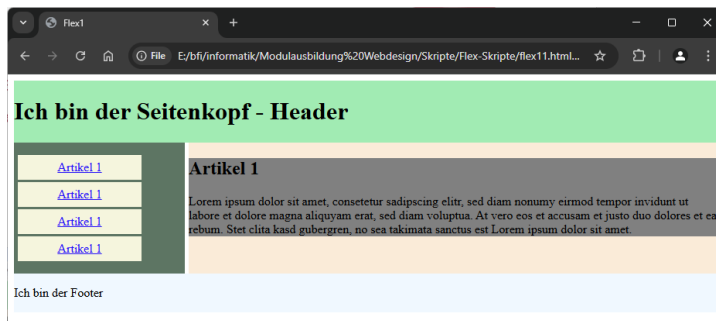


Abbildung 28: Der <body> wurde zum Flex-Container erklärt. Flex-Item (in Klammern die Zuordnung der Reihenfolge mit der Eigenschaft „order“) sind in diesem Beispiel die Elemente <header> (order: 1;), <nav> (order: 2;), <main> (order: 3;) und <footer> (order: 4;).

Es darf ein wenig experimentiert werden! In Abbildung 29 wurden die Formate der Elemente <nav> und <main> bearbeitet und darin die durch die Eigenschaft „order“ festgelegte Reihenfolge dieser beiden Elemente vertauscht. Der Menü-Block steht nun an dritter Stelle innerhalb des Flex-Containers, der Hauptteil <main> ist nun an der zweiten Position platziert. Beide Elemente passen nach wie vor in eine Zeile, wodurch sich lediglich deren Position in der Seite verändert.

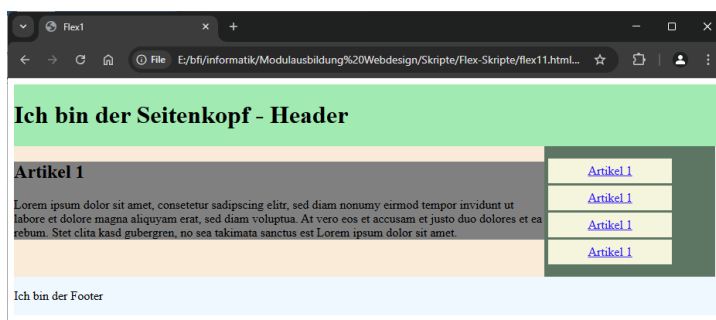


Abbildung 29: Die Werte der Order-Eigenschaft wurden für die Elemente <nav> und <main> vertauscht. Deswegen erscheint das Menü nun auf der rechten Seite. Der übrige HTML- sowie der CSS-Code sind unverändert.

Etwas eigenartig kommt die Webseite in der Abbildung 30 daher. An die Position drei wurde mithilfe der Eigenschaft „order“ das Element <footer> gesetzt, welches 100% der Flex-Container-Breite fordert.

⁷ Zur Erinnerung: Der Flex-Container wurde mit „flex-direction: row;“ und „flex-wrap: wrap;“ formatiert.

Damit passt es nicht mehr neben den Menü-Block und muss in die dritte Zeile rücken. Der <nav>-Container breitet sich hingegen auf der vollen Zeile aus, was an der dunkelgrünen Hintergrundfarbe ersichtlich ist. Ebenso verhält es sich mit dem <main>-Container, welcher hier an die Position 4 gesetzt wurde.

Natürlich ist das in Abbildung 30 gezeigte Beispiel nicht optimal, weil viel Platz auf der Seite leer bleibt und damit vergeudet wird.

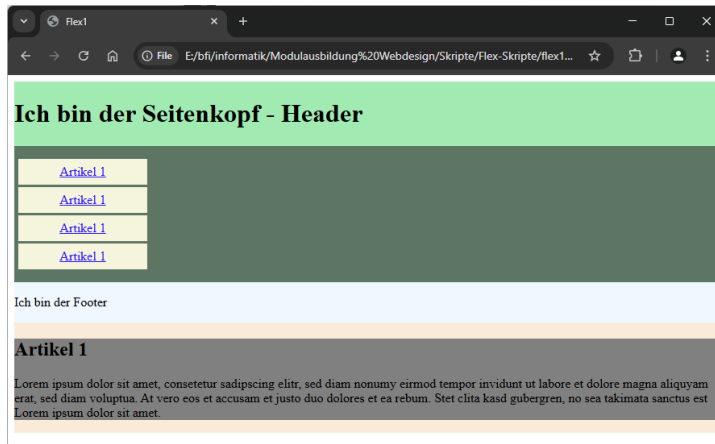


Abbildung 30: Die Reihenfolge der Flex-Item ist hier sehr ungeschickt gewählt worden, wodurch neben dem Menü viel Platz auf der Seite verschenkt wird. Ursache ist das Format des Flex-Items <footer> (hier mit „order: 3;“ vor dem <main>-Container platziert), der mit „flex: 100%;“ auf die volle Breite des Flex-Containers gesetzt wurde. Damit passt das Element nicht mehr in den freien Platz rechts neben dem Menü-Block.

Ein letztes Beispiel soll noch einmal die Wirkung der „order“-Eigenschaft verdeutlichen. Es sei betont, dass keinerlei Änderungen am HTML-Code (auch nicht in der Reihenfolge der Elemente) vorgenommen wurde. Im CSS-Code wurden lediglich die Werte der Eigenschaften „order“ verändert. In Abbildung 31 wurde dem <header> mit der Regel „order: 4;“ der Platz ganz unten in der Seite zugewiesen.



Abbildung 31: Völliger "Slapstick", aber lediglich in der Änderung der Order-Werte begründet, ist diese Anordnung der Webseite. Hier ist der <header> plötzlich in der Reihenfolge auf Platz 4. Auch die Order-Werte der anderen Elemente sind in diesem Beispiel verändert worden.

Verwendung der Flex-Technologie in Verbindung mit @media-Queries

Im Modul 13 - Responsive Webseiten – Medienabfragen wurden u.a. Abfragemöglichkeiten zur Höhe und Breite des Viewports vorgestellt. Mithilfe der @media-Abfrage können somit besondere Regeln festgelegt werden, die dann gelten sollen, wenn die Breite des Viewports möglicherweise zu schmal wird und die Elemente nicht mehr in der gewünschten Weise nebeneinander in die Seite passen. Die in diesem Modul gezeigten Verfahren erlauben es in solchen Fällen zum Beispiel die Reihenfolge bei der Darstellung der Elemente oder deren Breite zu verändern. Darüber hinaus können weitere CSS-Formate definiert werden, welche ausschließlich im Rahmen der Bedingung gelten, die mit der @media-Abfrage definiert wurde.

Wichtig: Bitte setzen Sie die `@media`-Abfragen grundsätzlich an das Ende des CSS-Codes. Achten Sie auch (und ganz besonders!) darauf, dass Sie in diesem Block lediglich Codezeilen schreiben oder verändern, die in diesem besonderen Ausnahmefall gelten sollen. Alle anderen Formate behalten weiterhin ihre Gültigkeit. Vermeiden Sie es also unbedingt, den kompletten zuvor geschriebenen CSS-Code in die `@media`-Blöcke zu kopieren! Das würde Ihren Code unübersichtlich und viel zu lang werden lassen. Im Fall einer Änderung besteht das Risiko, dass Sie in verschiedenen Situationen nicht mehr einheitliche Formate in der Seite verwenden.

Im folgenden Beispiel müssen nur die Formate dreier Elemente angepasst werden, um das gewünschte Layout (alle Elemente 100% der Flex-Container-Breite, Menü horizontal an vorletzter Stelle) zu erzielen.

```
@media (max-width: 750px) {  
  nav {  
    order: 3;  
    flex: 100%;  
    margin-right: 0px;  
    text-align: center;  
  }  
}
```

Die meiste Arbeit wird in der Neugestaltung des Menüs investiert. Es soll künftig an dritter Stelle – nach dem `<header>` und nach dem Hauptteil (`<main>`) erscheinen. Alle Elemente sollen die volle Breite des Viewports einnehmen. Dies wird mit der Regel „flex: 100%;“ gewährleistet, die jetzt (Viewports $\leq$ 750px) sowohl für das Element `<nav>` (bei Viewports $>750px$: 24%) als auch für das Element `<main>` (bei Viewports $>750px$: 75%) gelten sollen. Hier müssen also neue Werte für die Eigenschaft „flex“ vergeben werden.

Verändert wird für das Element `<nav>` auch der Wert für „margin-right“. Dieser wird auf 0px gesetzt. Die Inline-Inhalte werden mit der Regel „text-align: center;“ waagrecht mittig ausgerichtet. Dies ist deswegen möglich, weil im Regelblock mit dem Selektor „nav li“ – dieser war im regulären Code des Beispiels noch nicht enthalten – die Regel „display: inline-block;“ gesetzt wurde. Die List-Item erscheinen damit wie „Textzeichen“, also *ohne einen Zeilenumbruch*. Damit die Schaltflächen nicht unmittelbar aneinander „kleben“, wird auf der rechten Seite ein Außenabstand (*margin*) gesetzt.

```
nav li {  
  display: inline-block;  
  margin-right: 3px;  
}
```

Mit den beiden zuvor beschriebenen CSS-Regelblöcken wurde das Menü waagrecht über die volle Seitenbreite unterhalb⁸ des Hauptteils platziert. Abschließend sind zwei Anpassungen bei den Formaten des `<main>`-Elements erforderlich: Es soll an die zweite Position nach dem `<header>` rücken und sich in der vollen Breite des Viewports präsentieren. Das erledigt der folgende CSS-Block.

```
main {  
  order: 2;  
  flex: 100%  
}  
}
```

⁸ Selbstverständlich ist dies hier ein Übungsbeispiel um die Wirkung der Kombination von Flex-Layout und `@media`-Query zu veranschaulichen. Das Menü gehört normalerweise vor oder nach dem Header, in jedem Fall jedoch an eine gut zugänglichen Stelle der Webseite. Einige Webdesigner setzen zwei identische Menüblöcke vor und nach dem Hauptteil ein, wenn die Seite besonders lang ist.

Um die Formate zu prüfen, wird nun die (aktualisierte) Seite in einem Webbrowser aufgerufen und an der Seite schmaler bzw. breiter gezogen. An der Grenze von 750px der Viewport-Breite⁹ sollte sich die Reihenfolge sowie die Anordnung der Elemente verändern und die Seite ungefähr wie sie in der Abbildung 32 zu sehen ist, erscheinen.

Hinweis: In diesem Beispiel wird die horizontale Ausrichtung eines Menüs für das `<li>`-Element im Menü mit der Regel „`display: inline-block;`“ erreicht. Die horizontale Zentrierung erfolgt zudem mit der Regel „`text-align: center;`“. Was bei diesem Verfahren schwierig wird, ist die direkte vertikale Zentrierung der Elemente. Dieses Problem kann unter Umständen die nachfolgend gezeigte Alternative lösen.

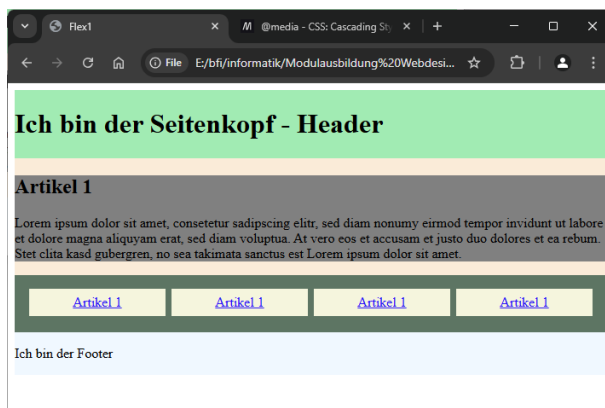


Abbildung 32: Bis zu einer Viewport-Breite von maximal 750px greifen in einem `@media`-Block definierte Regeln, mit denen einzelne Formate der Flex-Item-Elemente sowie deren Reihenfolge verändert werden. Die Seite kann somit responsiv auf Änderungen der Bildschirmbreite reagieren.

Ausrichtung eines Menü-Blocks

Durch die Deklaration eines HTML-Elements als *Flex-Container* werden die darin eingebundenen *Kind-Elemente* automatisch zu *Flex-Item*. Die Default-Einstellung für Flex ist dabei, dass alle Elemente von links nach rechts ausgerichtet werden. Doch Achtung: Es erfolgt ohne weitere Formatierung kein Umbruch am Rand des Flex-Containers! Wurde den Schaltflächen keine feste Breite zugewiesen, so werden diese sogar (ohne Umbruch) in ihrer Breite zusammengestaucht.

Soll ein Menü beispielsweise nicht – wie im ersten Modul noch gezeigt – senkrecht, sondern waagrecht ausgerichtet werden, dann kann dies sehr einfach umgesetzt werden, indem das Element `nav ul` als Flex-Container deklariert wird.

```
nav ul {
  list-style: none;
  padding-left: 0px;
  display: flex;
}
```

⁹ Dieser Wert gilt für das hier gezeigte Beispiel!

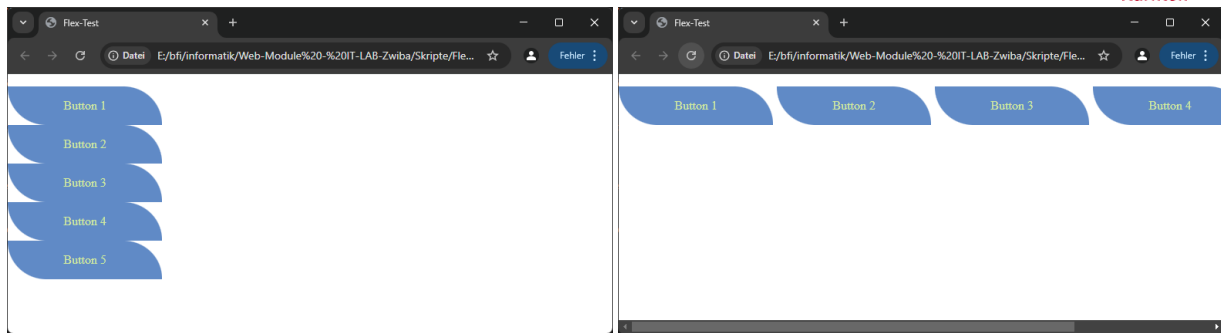


Abbildung 33: Links: Es wurde kein Flex-Container festgelegt. Rechts: Das Element `<ul>` (innerhalb des Containers `<nav>`) wurde mit der Regel „display: flex;“ zum Flex-Container deklariert. Die darin eingebundenen Kind-Elemente (in einer Liste sind es die List-Item, `<li>`). Sind nun die Flex-Item. Sie erscheinen nach der Grund-Formatierung eines Flex-Containers in horizontaler Ausrichtung. Wie zu sehen ist, passiert dies allerdings ohne einen Umbruch am Rand des Containers (den Schaltflächen wurde eine feste Breite zugewiesen).

Hinweis: Wie in diesem Modul bereits dargestellt wurde, kann ein Umbruch am Rand des Flex-Containers mit der Regel „flex-wrap: wrap;“ erzwungen werden (vgl. Abbildung 34).

Es sind nun noch kleinere „Schönheitsfehler“ zu korrigieren wie beispielsweise der Abstand der Elemente. Grundsätzlich würde sich aber empfehlen, ein besseres Design bei schmalen Viewports, in denen es zu Umbrüchen im Menü kommt, zu wählen. Zur Erinnerung: Entsprechende Entscheidungskriterien können mit den @media-Queries festgelegt und in deren Code-Blöcke passende Formate eingetragen werden.

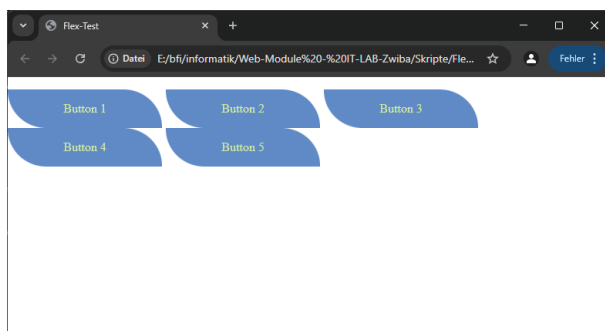


Abbildung 34: Das Format des Flex-Containers (hier: `nav ul`) wird um die Regel „flex-wrap: wrap;“ ergänzt. Jetzt werden die über den Rand des Containers herausreichenden Flex-Item in eine neue Zeile umbrochen.

Zentrierte Texte in einem Button

Es sei eine HTML-Seite mit einem kleinen Menü angenommen, wie es in den ersten Modulen beschrieben wurde. Die hellgrau geschriebenen CSS-Zeilen des folgenden Codes beschreiben einen Teil des Übergangs von der Liste zu Schaltflächen, welche hier (Selektor: `nav li`) in horizontaler Ausrichtung verwendet werden sollen. Der Regelblock mit dem Selektor `nav a` formatiert das Hyperlink-Element `<a>` innerhalb der List-Item als Schaltfläche. Dies ist bereits aus den früheren Lektionen bekannt.

```
nav ul {
    list-style: none;
    padding-left: 0px;
}

nav li {
    display: inline-block;
}
```


In der roten Zeile wird das Element `<a>` als Flex-Container festgelegt. Das Element `<a>` enthält hier lediglich einen Text, also kein weiteres Kind-Element, welches als Flex-Item zu deklarieren wäre. Dennoch können die CSS-Eigenschaften `justify-content` und `align-items` sinnvoll eingesetzt werden, um mit dem jeweiligen Wert „center“ für eine saubere horizontale UND vertikale Ausrichtung zu sorgen. Der Text – nicht mehr in weiteren Elementen eingepackt – nimmt hier die Stelle des Flex-Item ein.

Aufpassen! Auch hier sind Probleme in der Darstellung zu erwarten, wenn ein- und zweizeilige Schaltflächen gemischt werden!

```
nav a {
  color: #d1eba0;;
  background-color: #568ac5;
  display: flex;
  height: 30px;
  width: 200px;
  text-decoration: none;
  align-items: center;
  justify-content: center;
  border-radius: 0px 50px 0px 50px;
  margin-right: 5px;
}
```



Abbildung 35: Mit der Flex-Technologie können **Inhalte von Button** - beispielsweise in einem Menü - sehr einfach horizontal und vertikal zentriert ausgerichtet werden. Zweizeilige Beschriftungen der Schaltflächen, gemischt mit Einzeiligen erweisen sich aber nach wie vor als schwierig.

Reflexionsfragen

Bearbeiten Sie diese Fragen bitte wie immer in einer eigenen Text-Datei (z.B. MS-Word) und speichern Sie diese in den Modul-Ordner mit Ihrem Nachnamen und die Modulnummer als Dateiname ab!

Programmieraufgaben speichern Sie bitte in jeweils einem eigenen Unterverzeichnis. Bei Abgaben via Moodle komprimieren Sie die Dateien bitte in ein Zip-Archiv (keine anderen Kompressionsverfahren!).

- 14.1 Wie werden die beiden Achsen bezeichnet, an denen sich die Flex-Item ausrichten?
- 14.2 Erklären Sie die Begriffe „Eltern-Element“, „Kind-Element“ und „Nachfahre“ im Zusammenhang mit einer HTML-Struktur!
- 14.3 Welche CSS-Regel (Flex-Layout) müssen Sie wählen, um Elemente von rechts nach links horizontal anzuordnen?
- 14.4 Erklären Sie den Unterschied zwischen den CSS-Regeln „*flex-direction: row-reverse;*“ und „*flex-wrap: wrap-reverse;*“! Bearbeiten Sie falls erforderlich noch einmal den Themenschwerpunkt „Der Flex-Container“ ab Seite 5.
- 14.5 Sie haben die Eigenschaft „order“ kennengelernt. In welchem Teil eines Flex-Layouts verwenden Sie diese (Flex-Container, Flex-Item, Flex-Group)?
- 14.6 Was passiert, wenn Sie Elemente mit „order“ formatieren und zwei Elementen die gleiche Zahl als Wert zuweisen? Denken Sie auch an die Reihenfolge im HTML-Code!
- 14.7 An welcher Position werden Sie ein Element in einem Flex-Layout auf der Seite finden, wenn dieses (anders als alle anderen Elemente im Container nicht mit „order“ formatiert wurde)?
- 14.8 Programmieren Sie eine kurze HTML-Webseite mit einem Element `<section>`, welches Sie als Flex-Container definieren. In dieses Element sollen fünf `<div>`-Container mit jeweils einem Absatz enthalten sein, deren Inhalt eine einfache Nummerierung (1, 2, ...) sein soll. Formatieren Sie diese `<div>`-Container mit verschiedenen Hintergrundfarben und setzen Sie die Flex-Basis auf 150px. Die Flex-Item sollen jeweils individuelle Werte für „*flex-grow*“ zugewiesen bekommen: DIV1: 2, DIV2: 4, DIV3: 3, DIV4: 1, DIV5: 1.5 (Kein Komma). Errechnen Sie den „x-Faktor“ für eine Flex-Containerbreite von 1300px. Erstellen Sie eine Tabelle wie auf Seite 19 beschrieben und vergleichen Sie die Breiten der jeweiligen DIV-Container im Webbrowser.
- 14.9 Benutzen Sie die HTML- und CSS-Datei aus Aufgabe 14.8: Bei welcher Flex-Container-Breite sind **alle** Elemente so breit wie ihre Flexbasis.
- 14.10 Nehmen Sie sich noch einmal die Aufgabe aus 14.8 vor. Verwenden Sie wieder die gleichen Werte wie zuvor für „*flex-grow*“, dieses Mal jedoch für die Eigenschaft „*flex-shrink*“. Berechnen Sie den Faktor für eine Viewport-Breite von 750px und tragen Sie die Werte in eine Tabelle ein.
- 14.11 Programmieren Sie eine einfache Webseite mit einer ähnlichen Struktur, wie sie ab Seite 26 beschrieben wurde. Ergänzen Sie jedoch ein zusätzliches Flex-Item (`<aside>`), welches einen Anteil von 15% in der Breite des Flex-Containers haben soll. Reduzieren Sie die prozentuale Breite des im Beispiel gezeigten `<main>`-Elements so, dass die Elemente `<nav>`, `<main>` und `<aside>` ohne Umbruch in die Breite der Seite passen. Der `<header>` soll an Position 1 und der `<footer>` nun an Position 5 platziert werden.
- 14.12 Programmieren Sie einen ergänzenden CSS-Code für die Webseite aus Aufgabe 14.11, deren Viewport-Breite maximal 850 Bildschirmpunkte beträgt. In diesem Fall sollen **alle** Elemente über die volle Breite dargestellt werden. Das Menü soll waagrecht erscheinen, verwenden Sie hier diesmal bitte die Flex-Technologie für die horizontale sowie vertikal und horizontal zentrierte Darstellung der Schaltflächen. Stellen Sie sicher, dass bis maximal 850 Bildschirmpunkte die Elemente in der folgenden Reihenfolge erscheinen: `<header>`, `<nav>`, `<aside>`, `<main>` und `<footer>`